Gen-AI Hands-on 2 Unit 2

NAME: Kishore H N

SRN: PES2UG23CS278

SECTION: E

Abstract

This project serves as a comprehensive technical foundation for building LLM-powered applications using the **LangChain** framework. It transitions from basic API interaction to advanced architectural patterns, focusing on **Prompt Engineering**, **Reasoning Structures** (CoT, ToT, GoT), and **Retrieval-Augmented Generation (RAG)**. By abstracting model-specific code into modular pipelines, the system demonstrates how to create scalable, grounded, and "thoughtful" AI agents capable of complex logic and domain-specific knowledge retrieval.

Documentation: Systems and Architectures

1. LangChain Core & LCEL

The project utilizes **LangChain Expression Language (LCEL)** to replace "spaghetti code" with declarative pipelines.

- **Abstraction:** LangChain acts as a universal adapter, allowing developers to switch models (e.g., from OpenAI to Gemini) by changing a single line of code.
- **The Pipe Operator (|):** We built chains using the syntax Prompt | Model | Parser. This ensures that the output of one component—such as the **StrOutputParser**—cleanly feeds into the next, making the logic composable and easy to debug.

2. Prompt Engineering & In-Context Learning

We moved beyond "lazy" prompting to structured frameworks that manipulate the model's **Stochasticity** (randomness).

- **CO-STAR Framework:** We implemented prompts that define **C**ontext, **O**bjective, **S**tyle, **T**one, **A**udience, and **R**esponse format to eliminate ambiguity.
- **Few-Shotting:** By using FewShotChatMessagePromptTemplate, we built systems that learn patterns from examples without requiring mathematical fine-tuning.
- **Role Prompting:** The system distinguishes between **System Messages** (the rules), **Human Messages** (the input), and **AI Messages** (the history) to control the model's persona.

3. Advanced Reasoning Structures

To solve complex logic and math problems where standard models often fail, we built three distinct "thought" architectures:

- **Chain of Thought (CoT):** Forcing the model to "think step-by-step" to generate an inner monologue, which significantly improves accuracy in multi-step math problems.
- **Tree of Thoughts (ToT):** Implementing a branching system where the model generates multiple solutions, then acts as a "Judge" to select the most sustainable option.
- **Graph of Thoughts (GoT):** A networked approach where the model generates independent ideas (Divergence), aggregates the best parts of each (Convergence), and refines them into a final output.

4. RAG & Indexing (The Knowledge Layer)

To solve hallucinations and the "knowledge cutoff" problem, we built a **Retrieval-Augmented Generation** pipeline.

- **Embeddings:** We used HuggingFaceEmbeddings to convert text into 384-dimensional vectors, allowing the computer to understand semantic similarity via **Cosine Similarity** math.
- **Vector Storage (FAISS):** We implemented **FAISS** to store and search massive datasets efficiently.
- **Advanced Indexing:** We explored high-scale search algorithms, including **IVF** (clustering), **HNSW** (graph-based navigation), and **Product Quantization** (lossy compression for RAM efficiency).

1. [LangChain Foundation]

1_LangChain_Foundation.ipynb M | 2_Prompt_Engineering.ipynb M | .env

GenAI_Hands_on > GenAI-Hands-on > unit 2 > 1_LangChain_Foundation.ipynb > Unit 2 - Part 1a: LangChain Setup & Models > 7. Experiment: Consistency vs. Creativity

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | Python 3.11.9

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Define components
prompt = ChatPromptTemplate.from_template("The movie {movie_name} was released in what year? Calculate how many years ago that was from 2026."
parser = StrOutputParser()

# 2. The LCEL Chain (One-liner)
chain = prompt | llm | parser

# 3. Execution
print(chain.invoke({"movie_name": "Inception"}))
```

[14] ✓ 20.9s Python

```
The movie Inception was released in **2010**.

From 2026, that was **16 years ago** (2026 - 2010 = 16).
```

2. [Prompt Engineering Basics]

1_LangChain_Foundation.ipynb M | README.md | 2_Prompt_Engineering.ipynb M | .env

GenAI_Hands_on > GenAI-Hands-on > unit 2 > 2_Prompt_Engineering.ipynb > Unit 2 - Part 2a: The Anatomy of a Prompt > Assignment > structured_prompt = """

Generate | Code | Markdown | Interrupt | Restart | Clear All Outputs | Go To | Jupyter Variables | Outline | Python 3.11.9

Assignment

Write a structured prompt to generate a **Python Function**.

- **Context:** You are a Senior Python Dev.
- **Objective:** Write a function to reverse a string.
- **Constraint:** It must use recursion (no slicing `[::-1]`).
- **Style:** Include detailed docstrings.

```python
structured_prompt = """
# Context
You are a Senior Python Developer who prioritizes clean, academic code.

# Objective
Write a Python function named 'reverse_string' that takes a string as input.

# Constraints
1. You MUST use recursion.
2. Do NOT use slicing like [::-1].
3. Include a detailed Google-style docstring.

# Output Format
Provide only the Python code block.
"""

print(llm.invoke(structured_prompt).content)
```

[4] ✓ 7.1s Python

Output:

```python
def reverse_string(text: str) -> str:
    """Reverses a given string using a recursive approach.

    This function takes an input string and returns a new string with its
    characters in reverse order. It operates by recursively processing
    the substring excluding the first character, and then appending the
    first character to the result of that recursive call. This process
    unwinds, effectively building the reversed string from right to left.

    The base cases for the recursion are an empty string or a string
    containing a single character, both of which are considered already
    reversed and are returned as is.

    Args:
        text: The input string to be reversed. Must be a string.

    Returns:
        A new string with the characters of the input string in reverse order.

    Examples:
        >>> reverse_string("hello")
        'olleh'
        >>> reverse_string("Python")
        'nohtyP'
        >>> reverse_string("")
        ''
        >>> reverse_string("a")
        'a'
        >>> reverse_string("madam")
```

```
        'madam'
    """
    # Base case: An empty string or a string with a single character
    # is already reversed.
    if not text or len(text) == 1:
        return text
    else:
        # Recursive step:
        # 1. Take the substring starting from the second character (text[1:]).
        # 2. Recursively reverse this substring.
        # 3. Append the first character of the original string (text[0])
        #    to the result of the recursive call.
        return reverse_string(text[1:]) + text[0]
```

3. [Advanced Prompting]

3. The Experiment: A Tricky Math Problem

Let's try a problem that requires multi-step logic.

Problem: "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?"

```python
question = "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?"

# 1. Standard Prompt (Direct Answer)
prompt_standard = f"Answer this question: {question}"
print("--- STANDARD (Llama3.1-8b) ---")
print(llm.invoke(prompt_standard).content)
```

✓ 0.5s Python

```
--- STANDARD (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to add the initial number of tennis balls he had (5) to the number of tennis balls he bou

2 cans * 3 tennis balls per can = 6 tennis balls

Now, let's add the initial number of tennis balls (5) to the number of tennis balls he bought (6):

5 + 6 = 11

So, Roger now has 11 tennis balls.
```

Critique

Smaller models often latch onto the visible numbers (5 and 2) and simply add them (7), ignoring the multiplication step implied by "cans".

Let's force it to think.

```python
# 2. CoT Prompt (Magic Phrase)
prompt_cot = f"Answer this question. Let's think step by step. {question}"

print("--- Chain of Thought (Llama3.1-8b) ---")
print(llm.invoke(prompt_cot).content)
```

✓ 0.2s Python

```
--- Chain of Thought (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to follow these steps:

1. Roger already has 5 tennis balls.
2. He buys 2 more cans of tennis balls. Each can has 3 tennis balls, so he buys 2 x 3 = 6 more tennis balls.
3. Now, we add the tennis balls he already had (5) to the tennis balls he bought (6). 5 + 6 = 11

So, Roger now has 11 tennis balls.
```

4. [RAG & Vector Stores]

How do we know if two vectors are close? We measure the **Angle** between them.

Cosine Similarity Formula

$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

- **1.0**: Arrows point in the Exact Same Direction (Identical).
- **0.0**: Arrows are Perpendicular (Unrelated).
- **-1.0**: Arrows point in Opposite Directions (Opposite).

Experiment: Let's compare "Cat", "Dog", and "Car".

```python
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

vec_cat = embeddings.embed_query("Cat")
vec_dog = embeddings.embed_query("Dog")
vec_car = embeddings.embed_query("Car")

print(f"Cat vs Dog: {cosine_similarity(vec_cat, vec_dog):.4f}")
print(f"Cat vs Car: {cosine_similarity(vec_cat, vec_car):.4f}")
```

✓ 0.0s

```
Cat vs Dog: 0.6606
Cat vs Car: 0.4633
```